


# Project Nurse Simulator

## DAW 2
## 2026/2027

pr5

Ignacio Breñas Muñoz
Christopher Roger Tito Farez
Zehao Yin
Gorka Ramirez

stucom

## Contenido

1. Explicación del proyecto ..... 3
1.1. Descripción general ..... 3
1.2. Qué cambia respecto a la pr04 ..... 3
2. Cómo organizamos el trabajo ..... 4
2.1. Metodología con GitHub ..... 4
2.2. Evidencias de la organización ..... 5
Tablero de GitHub Project ..... 5
Issues y ramas ..... 6
Pull requests y commits ..... 6
3.Operaciones Crud ..... 7
3.1. Operación Create: POST /nurse (Gorka Ramirez) ..... 7
3.2. Operación Read: GET /nurse/{id} (Christopher Roger Tito) ..... 7
3.3. Operación Update: PUT /nurse/{id} (Zehao Yin) ..... 7
3.4. Operación Delete: DELETE /nurse/{id} (Ignacio Breñas) ..... 7
3.5. Resumen del reparto de tareas en el repositorio ..... 8
3.6. Integración y unificación en NurseController ..... 8
4. Integración continua (GitHub Actions) ..... 9
4.1. Pruebas unitarias para NurseController (PHPUnit 11) ..... 9
4.2. Ejecución automática con éxito en Pull Request ..... 10
4.3. Simulación de fallo en pruebas unitarias y detección por el pipeline ..... 11
4.4. Workflows adicionales de CI ..... 13
5. Documentación del proyecto (README.md en GitHub) ..... 14
6. Comprobación con Postman ..... 15
6.1. POST /nurse (Create) ..... 15
6.2. GET /nurse/{id} (Read) ..... 18
6.3. PUT /nurse/{id} (Update) ..... 21
6.4. DELETE /nurse/{id} (Delete) ..... 22
6.5. Endpoints ya existentes ..... 25

https://github.com/Ignaciobrenas/Nurse-Simulator

# 1. Explicación del proyecto

## 1.1. Descripción general

https://github.com/Ignaciobrenas/Nurse-Simulator

Nurse Simulator es una API REST que simula la gestión de enfermeros de un centro hospitalario. Está desarrollada con el framework Symfony 7.4 sobre PHP 8.2 y utiliza una base de datos PostgreSQL alojada en Supabase como motor de persistencia relacional con Doctrine ORM.

Este documento recoge la entrega de la pr05: Back-end CRUD e Integración Continua (DAM MP0616). En esta práctica el proyecto evoluciona desde un modelo de consulta a un sistema transaccional completo: se implementan las cuatro operaciones del CRUD (Create, Read by ID, Update, Delete) sobre la entidad Nurse, y se añade un ecosistema de integración continua (CI) en GitHub Actions para asegurar automáticamente la calidad del código, las pruebas unitarias y la seguridad en cada Pull Request.

## 1.2. Qué cambia respecto a la pr04

En la pr04 conectamos la aplicación a la base de datos PostgreSQL en Supabase mediante Doctrine, pero únicamente disponíamos de endpoints de lectura global y login.

En la pr05 el proyecto incorpora las mutaciones completas de datos con hashing seguro y validación estricta, junto con la automatización de pruebas:

| Ámbito / Endpoint | Antes (pr04) | Ahora (pr05) |
|---|---|---|
| POST /nurse (Create) | No implementado. Inserción solo manual o fixtures. | Alta completa con validación de campos, formato email y contraseña cifrada con Bcrypt. |
| GET /nurse/{id} (Read) | No disponible. Solo búsqueda por nombre o listado total. | Lectura por ID numérico con control de 404 Not Found y validación de enteros. |
| PUT /nurse/{id} (Update) | No disponible. | Modificación total o parcial, detección de 204 No Content si no hay cambios y control de email duplicado (409). |
| DELETE /nurse/{id} (Delete) | No disponible. | Borrado físico seguro con verificación previa de existencia (404 si no existe). |

Además, a nivel de infraestructura se integraron 4 flujos de GitHub Actions (PHPUnit con 59 tests y 139 aserciones, Postman Newman E2E, calidad de código PSR-12 y auditoría de seguridad Trivy).

# 2. Cómo organizamos el trabajo

## 2.1. Metodología con GitHub

El equipo mantiene la misma metodología de trabajo que en las prácticas anteriores, coordinada por Ignacio Breñas como Tech Lead y Project Manager:

- Cada tarea es un issue, gestionado en un tablero de GitHub Project con las columnas Todo, In progress y Done.
- Cada issue tiene su rama propia (<nº>-<descripción>), asegurando un entorno de trabajo aislado para cada integrante.
- Los cambios entran obligatoriamente por Pull Request, con revisión técnica previa y paso de los workflows de CI antes de mergear hacia staging y finalmente main.
- Los commits siguen estrictamente la convención usuario/Tipo: descripción #issue (por ejemplo, ibrenas/Feat: implement DELETE /nurse/{id} endpoint #57).

# 2.2. Evidencias de la organización

## Tablero de GitHub Project

Estado de las tareas de la pr05 en el tablero del proyecto Kanban.

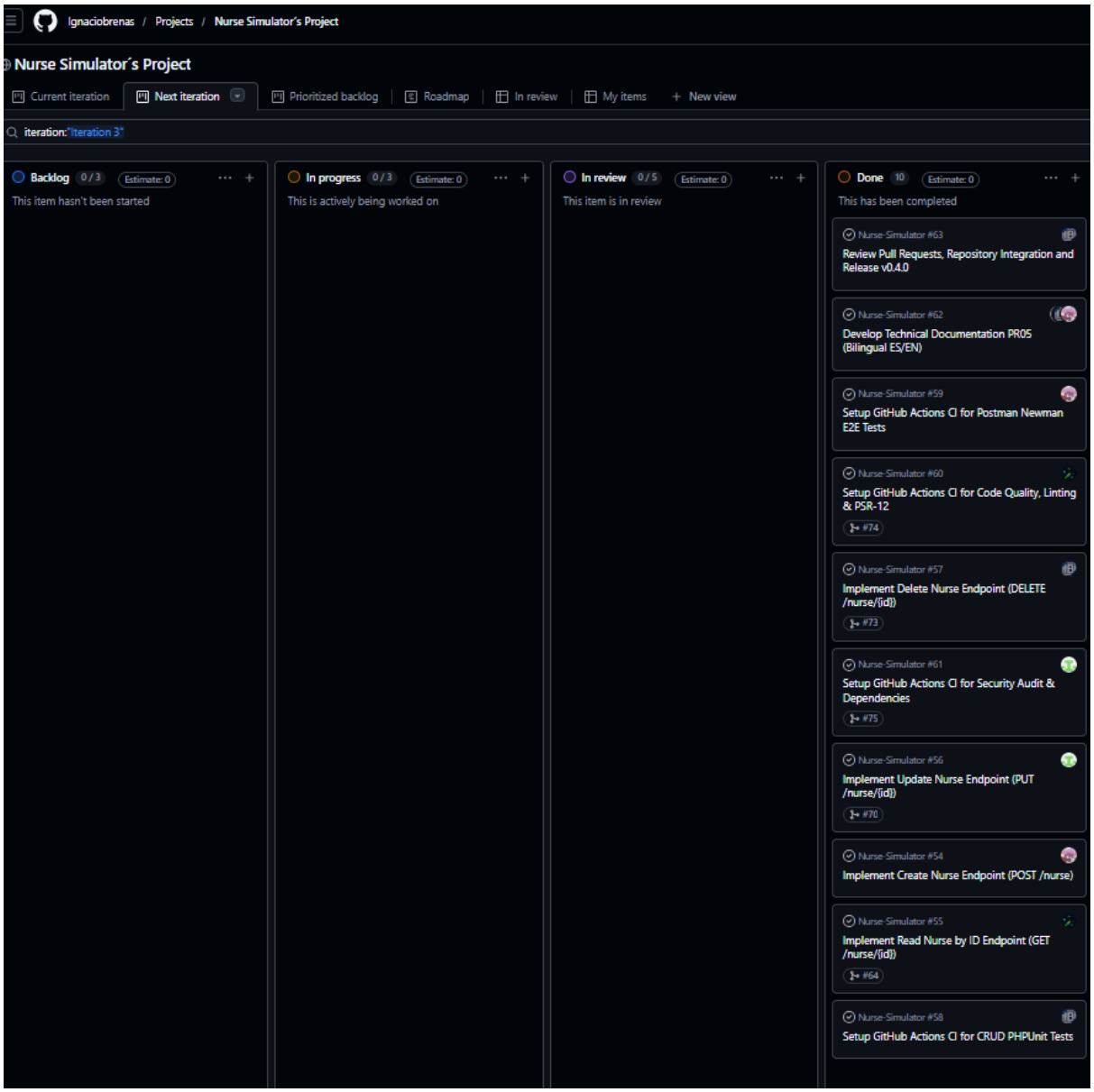



Figura 1. Tablero de tareas de la pr05.

## Issues y ramas

Listado de issues de la pr05 y su rama asociada.

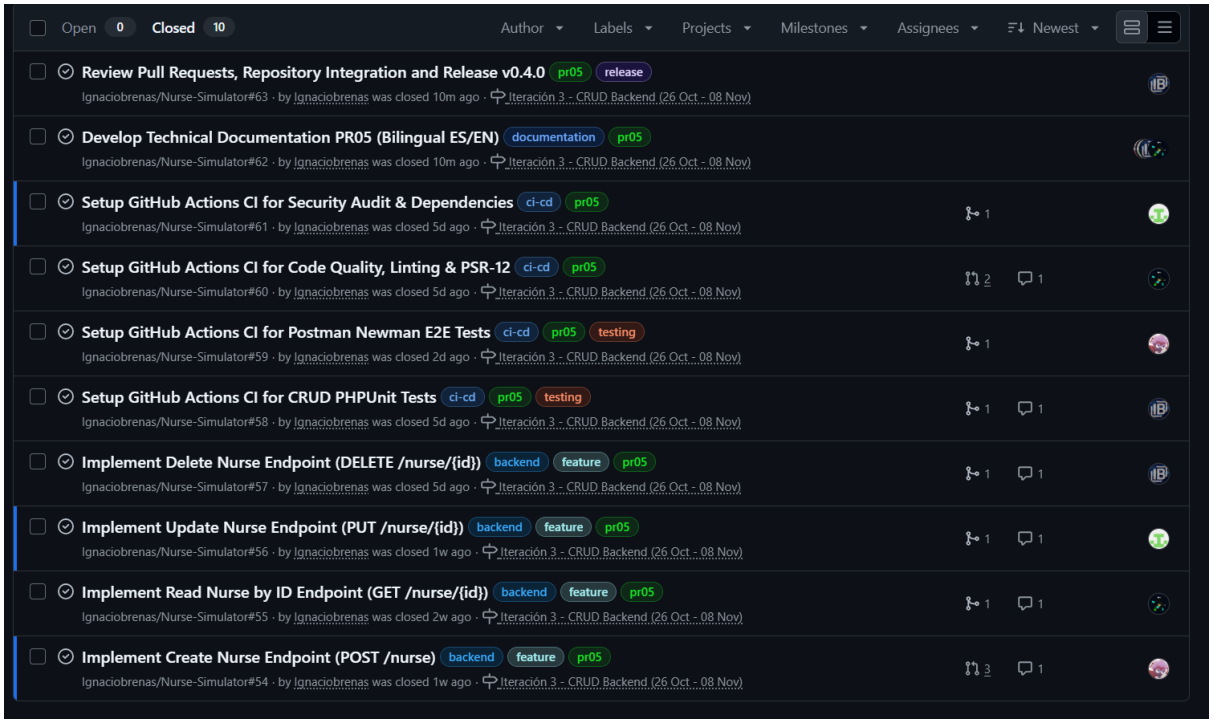



Figura 2. Issues y ramas de la pr05.

## Pull requests y commits

Pull requests revisadas y fusionadas, con el historial de commits y trazabilidad.




Figura 3. Pull requests de la pr05.

# 3.Operaciones Crud

El desarrollo de las cuatro operaciones CRUD sobre la entidad Nurse y la configuración de los flujos de Integración Continua en GitHub Actions se distribuyó entre los miembros del equipo dentro del repositorio del proyecto. Cada integrante asumió la responsabilidad técnica de una operación (controlador, validaciones, códigos de respuesta HTTP y pruebas unitarias):

## 3.1. Operación Create: POST /nurse (Gorka Ramirez)

Gorka desarrolló el endpoint de creación de nuevos enfermeros (Issue #54). El controlador recibe los datos en formato JSON, valida que estén presentes todos los campos obligatorios (name, email, password), comprueba mediante expresión regular que el email tenga una estructura válida (RFC 5322), y consulta la base de datos para certificar que el correo no esté ya en uso, respondiendo 409 Conflict en caso de duplicidad. La contraseña se hashea de forma segura con el algoritmo Bcrypt (cost 12) antes de persistir la entidad. Asimismo, Gorka configuró el pipeline de pruebas E2E de API con Postman y Newman (Issue #59).

## 3.2. Operación Read: GET /nurse/{id} (Christopher Roger Tito)

Christopher implementó la consulta de enfermero por su identificador primario (Issue #55). Si el ID facilitado no es estrictamente numérico devuelve 400 Bad Request; si el registro no existe en la base de datos devuelve 404 Not Found con mensaje descriptivo en JSON; y si existe retorna un 200 OK con los datos completos del enfermero (excluyendo el hash de contraseña). En el apartado de CI, Christopher integró la comprobación automatizada de calidad de código y formato PSR-12 con PHP-CS-Fixer (Issue #60).

## 3.3. Operación Update: PUT /nurse/{id} (Zehao Yin)

Zehao se encargó de la modificación de datos de enfermeros existentes (Issue #56). Permite actualizar name, email y password tanto de forma total como parcial. Si se envía una nueva contraseña, esta se cifra automáticamente con Bcrypt. Si la petición no contiene cambios respecto al estado actual del registro, responde 204 No Content. Controla la no duplicidad de correo (409 Conflict) y el recurso no encontrado (404 Not Found). En CI, Zehao implementó el workflow de auditoría de seguridad y análisis de dependencias mediante Trivy (Issue #61).

## 3.4. Operación Delete: DELETE /nurse/{id} (Ignacio Breñas)

Ignacio implementó el endpoint de eliminación física de registros (Issue #57). Verifica primero si el enfermero existe en la base de datos: si existe, procede a su borrado a través del EntityManager de Doctrine y confirma la operación con 200 OK; si no existe devuelve 404 Not Found. Además, valida el parámetro de ruta descartando valores alfanuméricos con 400 Bad Request. En CI, Ignacio configuró el pipeline principal con la suite de pruebas unitarias y de integración de endpoints CRUD con PHPUnit 11 (Issue #58).

## 3.5. Resumen del reparto de tareas en el repositorio

En el repositorio oficial del proyecto se puede comprobar con claridad el desarrollo colaborativo de cada operación CRUD mediante sus ramas dedicadas, los Pull Requests validados y la ejecución de los flujos de CI asociados:

| Integrante | Operación CRUD & Endpoint | Issue & Rama en GitHub | Workflow de CI (GitHub Actions) |
|---|---|---|---|
| Gorka Ramirez | POST /nurse (Create) | Issue #54 (54-feat-create-nurse) | Postman / Newman E2E (Issue #59, PR #77) |
| Christopher Roger Tito | GET /nurse/{id} (Read) | Issue #55 (55-feat-read-nurse) | Calidad de código PSR-12 (Issue #60, PR #68) |
| Zehao Yin | PUT /nurse/{id} (Update) | Issue #56 (56-feat-update-nurse) | Auditoría seguridad Trivy (Issue #61, PR #75) |
| Ignacio Breñas Muñoz | DELETE /nurse/{id} (Delete) | Issue #57 (57-feat-delete-nurse) | Pruebas unitarias PHPUnit (Issue #58, PR #74) |

## 3.6. Integración y unificación en NurseController

Una vez desarrolladas y probadas las ramas de cada endpoint, se integraron mediante Pull Requests en la rama principal del repositorio del proyecto:

https://github.com/Ignaciobrenas/Nurse-Simulator

En el repositorio se unificaron las 4 rutas bajo el controlador NurseController.php, estandarizando las respuestas JSON, los códigos de error HTTP y el formato de los mensajes.

Asimismo, se unificó la lógica de cifrado Bcrypt tanto en el registro (create), actualización (update) y validación de credenciales (login), garantizando una compatibilidad absoluta con la base de datos PostgreSQL en Supabase.

# 4. Integración continua (GitHub Actions)

Para asegurar que cada Pull Request mantenga la estabilidad del proyecto y no rompa ninguna funcionalidad, implementamos un ecosistema de integración continua con 4 workflows automatizados en GitHub Actions.

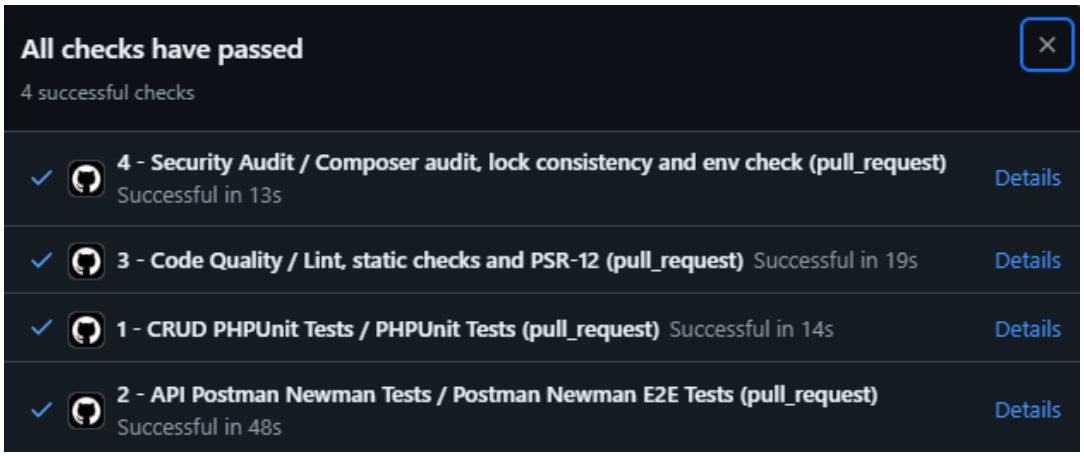
## 4.1. Pruebas unitarias para NurseController (PHPUnit 11)

Se desarrolló una suite completa de pruebas unitarias y de integración en tests/Controller/NurseControllerTest.php, formada por 59 pruebas y 139 aserciones que verifican de manera exhaustiva todos los métodos del controlador:

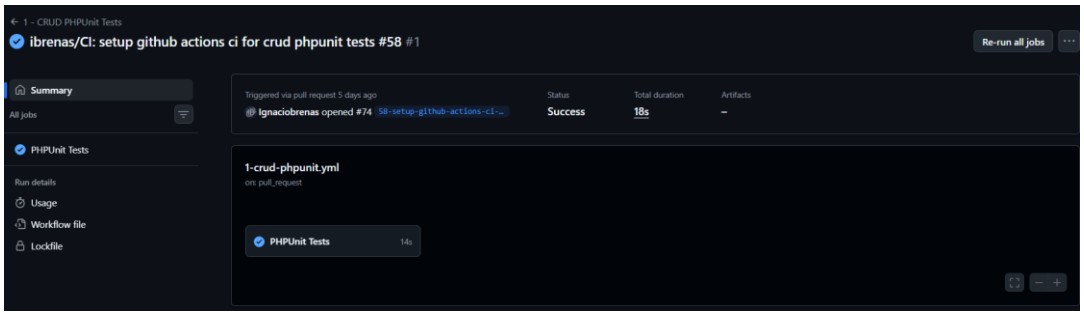
- Casos de éxito: Creación (201 Created), Lectura por ID (200 OK), Actualización con cambios (200 OK) y sin cambios (204 No Content), Eliminación (200 OK), Listado (200 OK), Login (200 OK) y Búsqueda por nombre (200 OK).
- Casos de error: Peticiones incompletas o mal formadas (400 Bad Request), credenciales erróneas (401 Unauthorized), registros inexistentes (404 Not Found), colisión de email duplicado (409 Conflict) y formato de email inválido (422 Unprocessable Content).

## 4.2. Ejecución automática con éxito en Pull Request

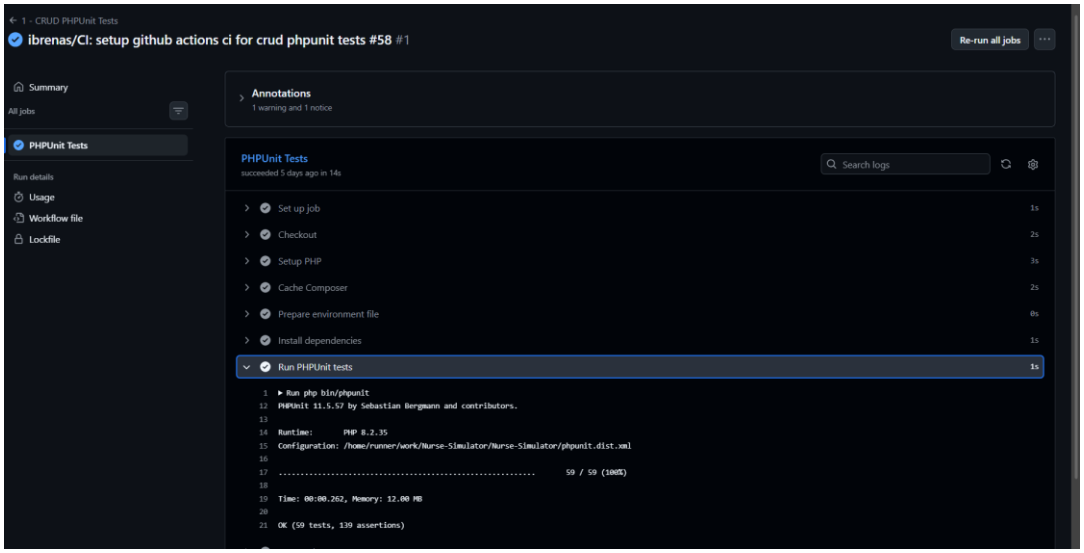
Cuando una Pull Request contiene código correcto que supera todas las aserciones, el pipeline de GitHub Actions se ejecuta automáticamente y reporta el estado en verde (Passed), habilitando la fusión segura hacia staging o main:




*CI PHPUnit: Pull Request con pipeline en verde (Passed)*



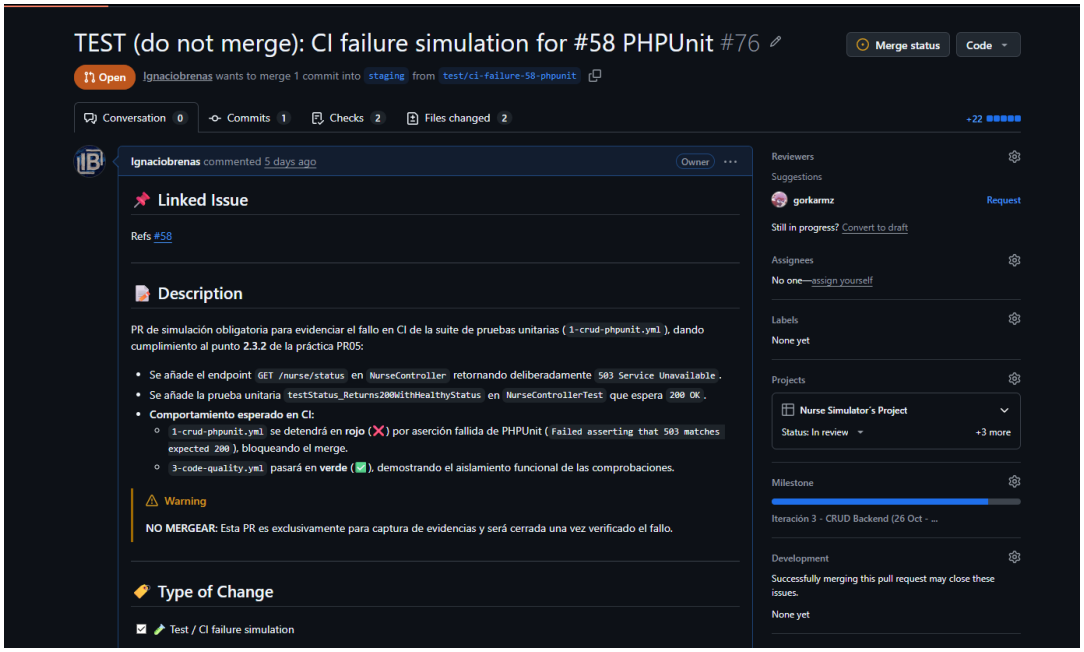

*CI PHPUnit: Resumen satisfactorio (59 tests, 139 assertions)*



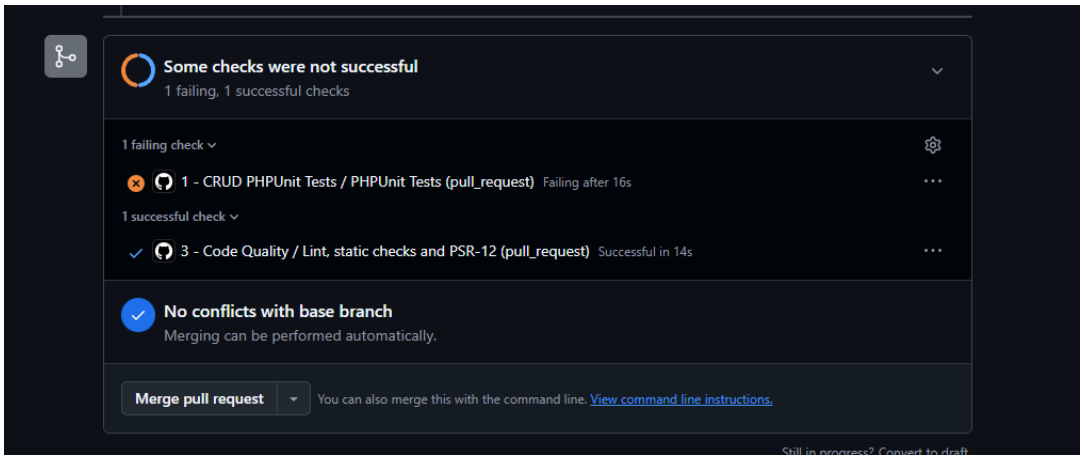

*CI PHPUnit: Detalle de pasos completados en el runner*

## 4.3. Simulación de fallo en pruebas unitarias y detección por el pipeline

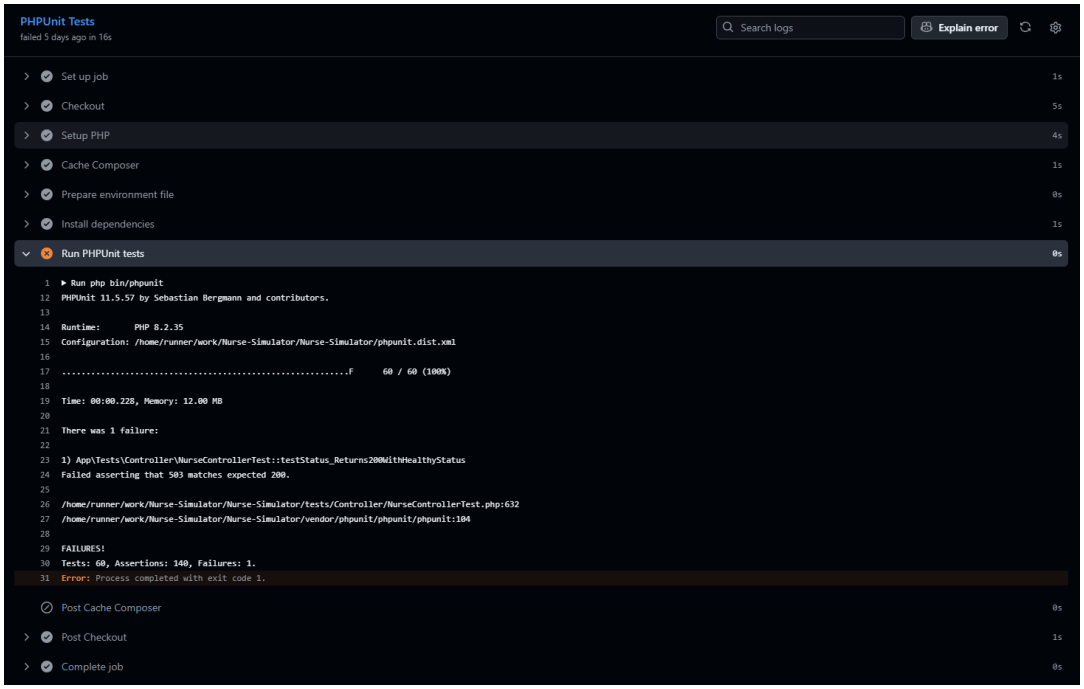
Para demostrar que el sistema de CI protege de forma activa la rama principal, se forzó deliberadamente un fallo en una de las aserciones de prueba en una Pull Request. El pipeline detectó de inmediato el error, detuvo la integración y bloqueó el merge:




*CI PHPUnit: Pull Request bloqueada por fallo detectado*




*CI PHPUnit: Comprobación fallida que impide la fusión*




*CI PHPUnit: Detalle del error reportado en la consola del pipeline*

## 4.4. Workflows adicionales de CI

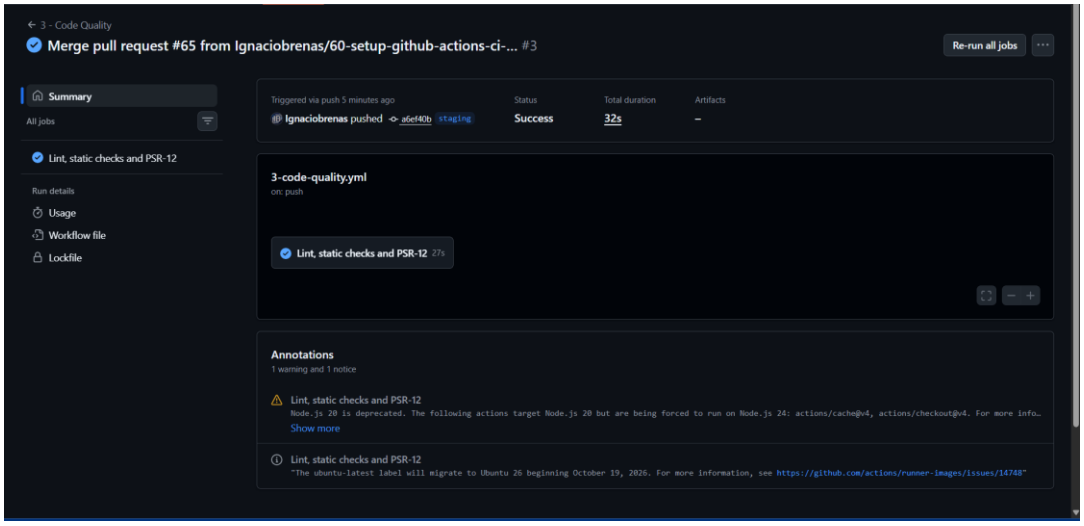
El proyecto cuenta además con otros 3 flujos automatizados de CI:

- API Postman Newman (2-api-postman.yml): Ejecución automatizada de colecciones de Postman contra la base de datos real.



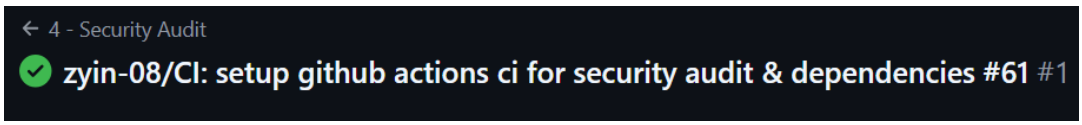

*CI Postman Newman: Pipeline E2E validando endpoints en verde*

- Calidad de Código PSR-12 (3-code-quality.yml): Análisis estático con PHP-CS-Fixer asegurando el cumplimiento estricto del estándar de formato.




*CI Code Quality: Verificación de estándares PSR-12*

- Auditoría de Seguridad (4-security-audit.yml): Detección de vulnerabilidades en dependencias y comprobación de ausencia de secretos con Trivy.




*CI Security Audit: Escaneo de vulnerabilidades superado*

# 5. Documentación del proyecto (README.md en GitHub)

El archivo README.md del repositorio principal ha sido completamente renovado, cumpliendo minuciosamente con los 4 apartados obligatorios requeridos por la rúbrica:

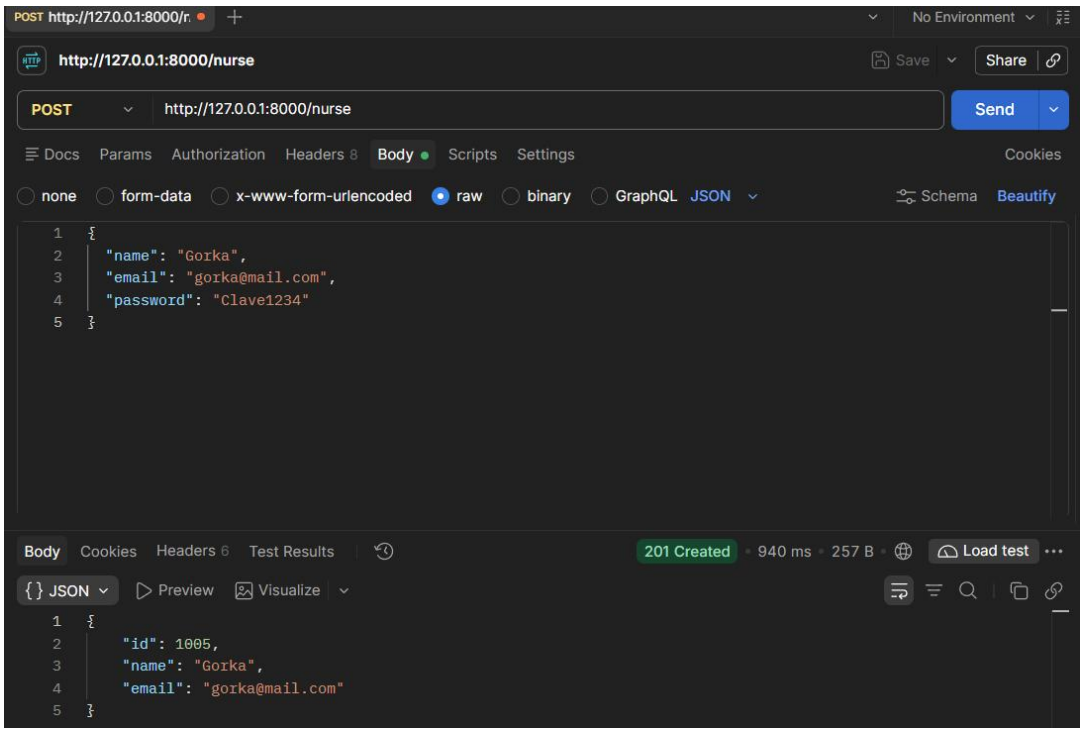
- 1. Título del proyecto: Identificación de la práctica con insignias (badges) en tiempo real que reflejan el estado de los 4 pipelines de CI.
- 2. Descripción: Resumen técnico del sistema, stack utilizado (Symfony 7.4, PHP 8.2, PostgreSQL en Supabase, Bcrypt, PHPUnit 11) y arquitectura del backend.
- 3. Instalación: Guía detallada para clonar el proyecto, instalar paquetes con Composer, configurar credenciales en .env.local y ejecutar migraciones de Doctrine.
- 4. Uso: Documentación completa de cada endpoint con método HTTP, ruta, tabla de respuestas y ejemplos listos para copiar y ejecutar con cURL.

# 6. Comprobación con Postman

A continuación se presentan las evidencias de funcionamiento de todos los endpoints de la API interactuando directamente contra la base de datos de PostgreSQL en Supabase:

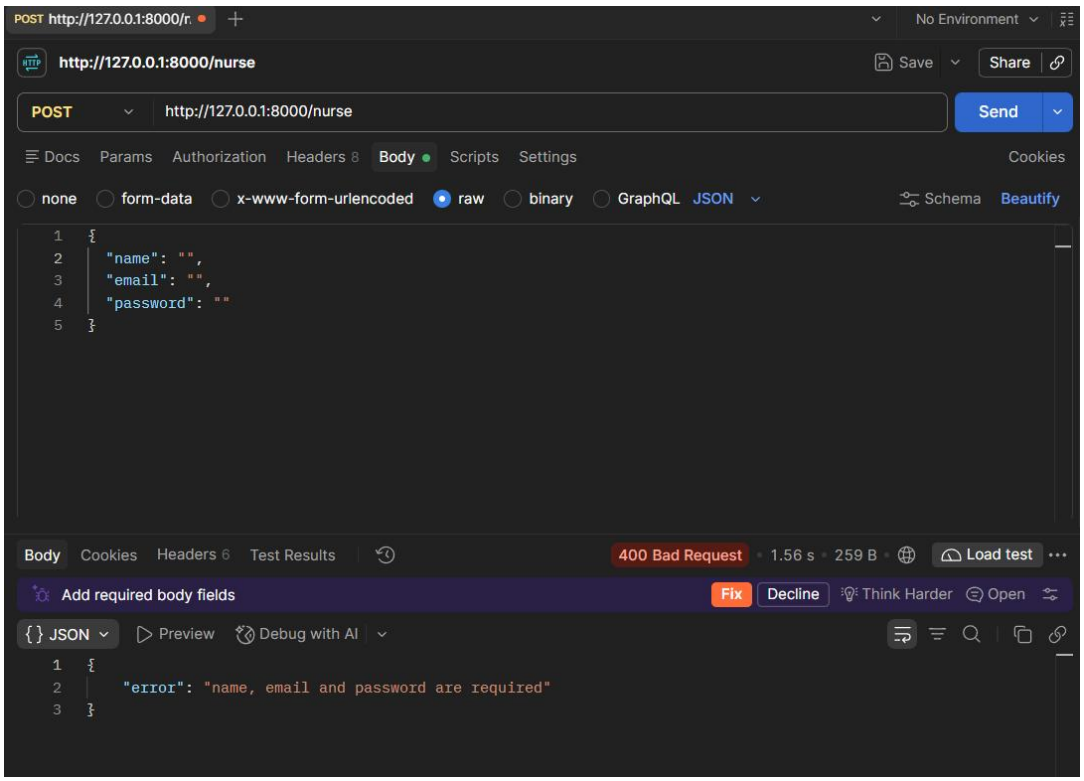
## 6.1. POST /nurse (Create)

Creación satisfactoria de un nuevo enfermero con contraseña encriptada en Bcrypt (201 Created).



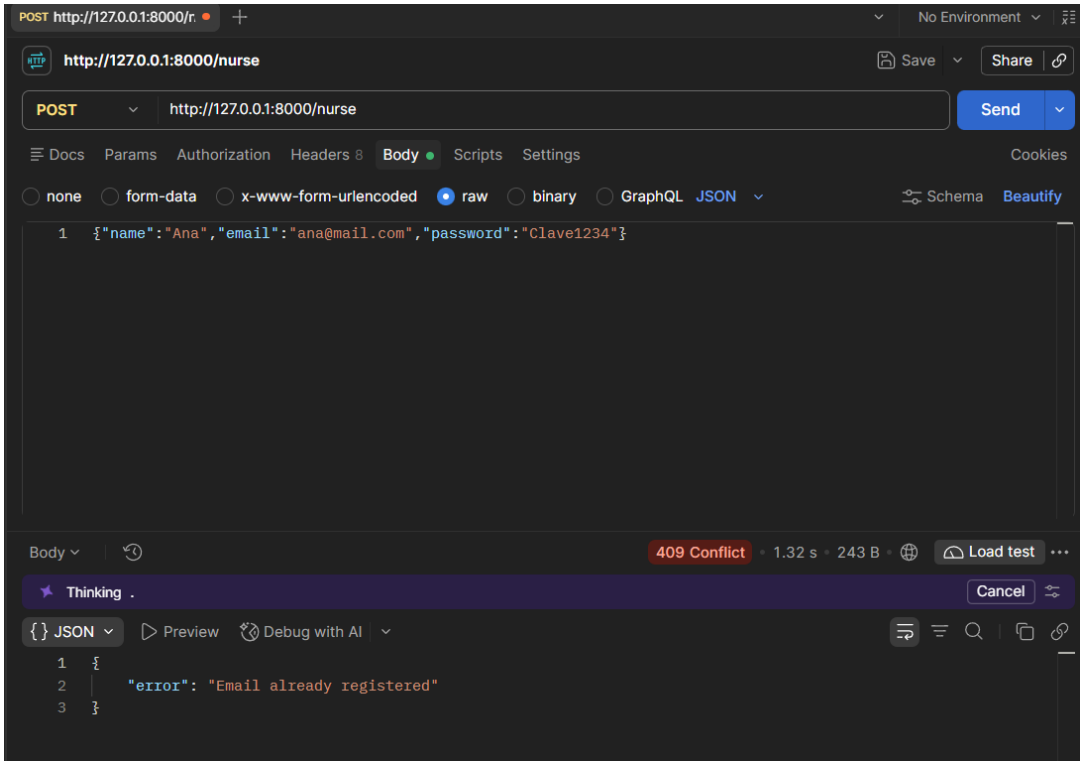

*Postman Create (201 Created) 1*

Petición rechazada al omitir campos obligatorios en el JSON (400 Bad Request).



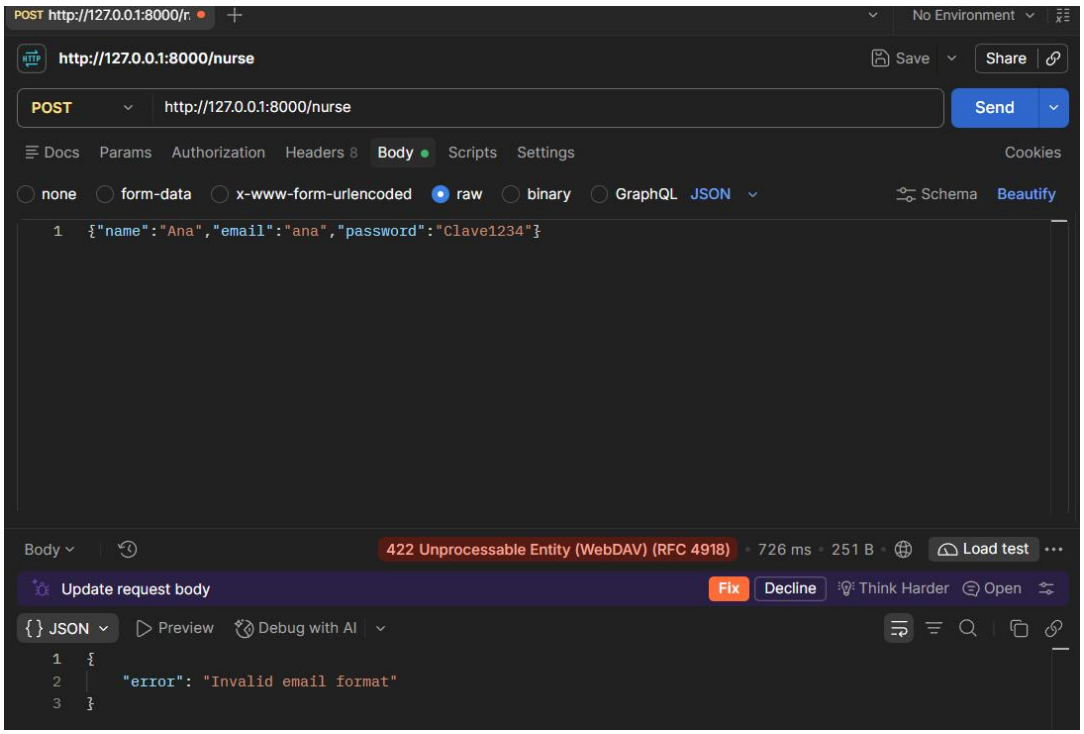

*Postman Create (400 Bad Request required) 1*

Control de duplicidad: rechazo al registrar un correo que ya existe en la base de datos (409 Conflict).




*Postman Create (409 Conflict) 1*

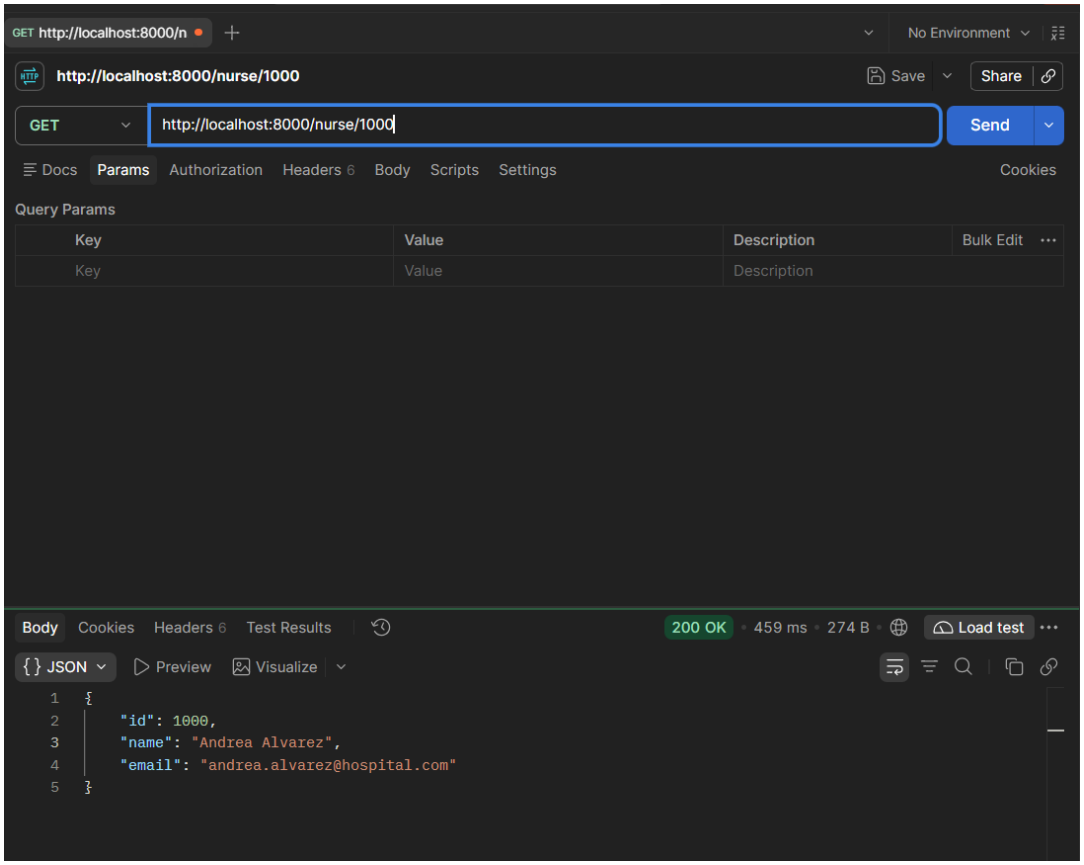
Validación de formato sintáctico de email inválido (422 Unprocessable Content).




*Postman Create (422 Unprocessable Content Email) 1*

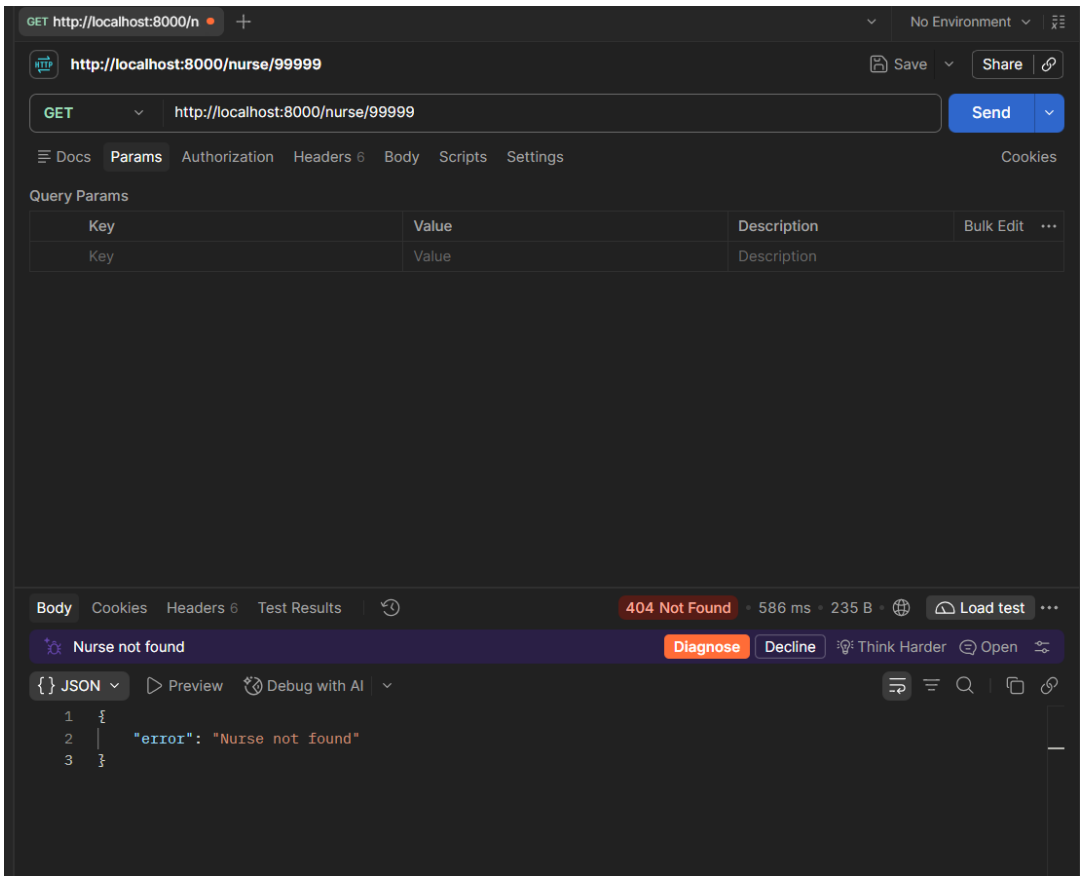
## 6.2. GET /nurse/{id} (Read)

Consulta por identificador primario numérico existente (200 OK).



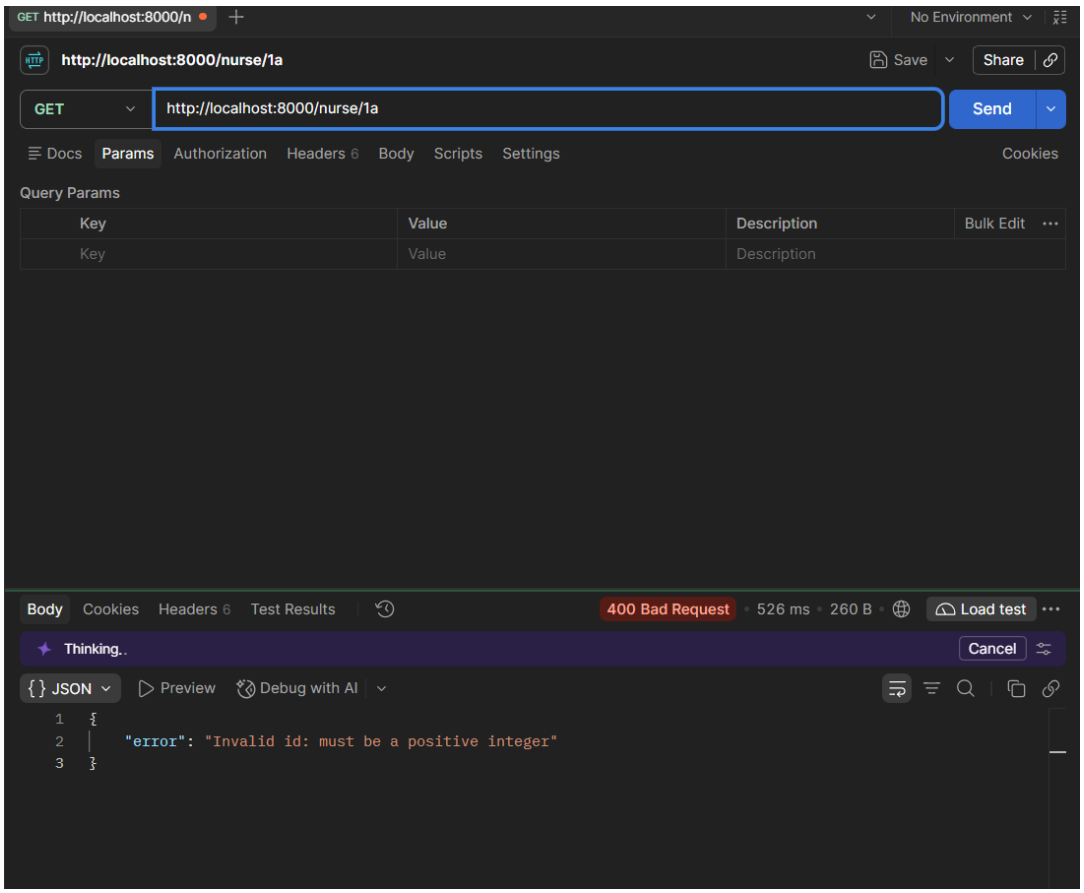

*Postman Read by ID (200 OK) 1*

Respuesta al consultar un identificador que no existe en la base de datos (404 Not Found).




*Postman Read by ID (404 Not Found) 1*

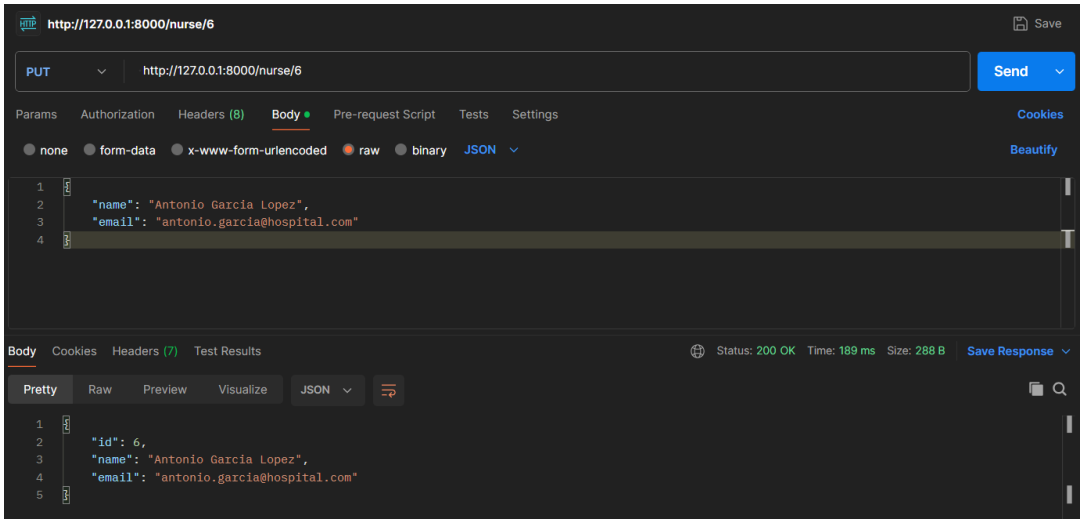
Validación de parámetro de ID no numérico (400 Bad Request).




*Postman Read by ID (400 Bad Request) 1*

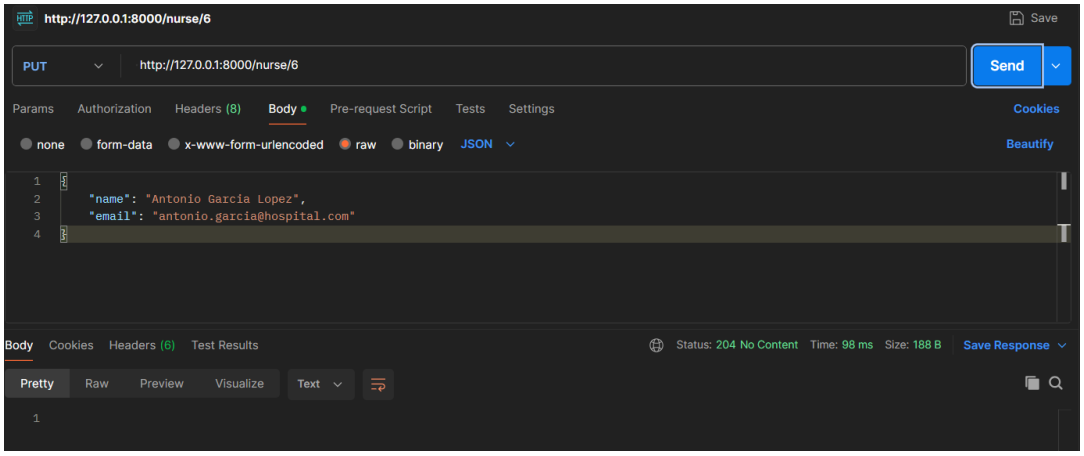
## 6.3. PUT /nurse/{id} (Update)

Actualización exitosa de campos de un enfermero (200 OK).



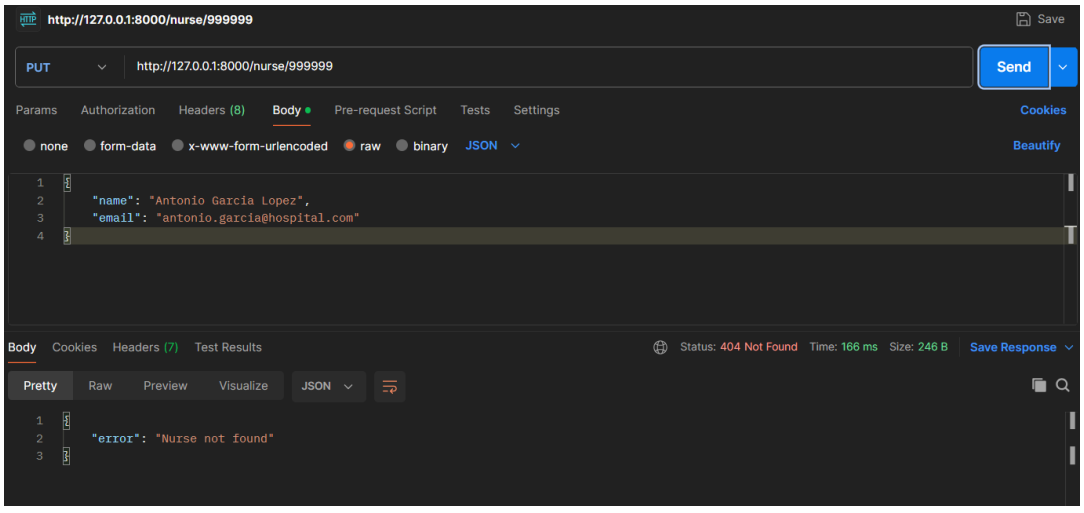

*Postman Update (200 OK) 1*

Optimización de respuesta cuando no se detectan cambios que persistir (204 No Content).




*Postman Update (204 No Content) 1*

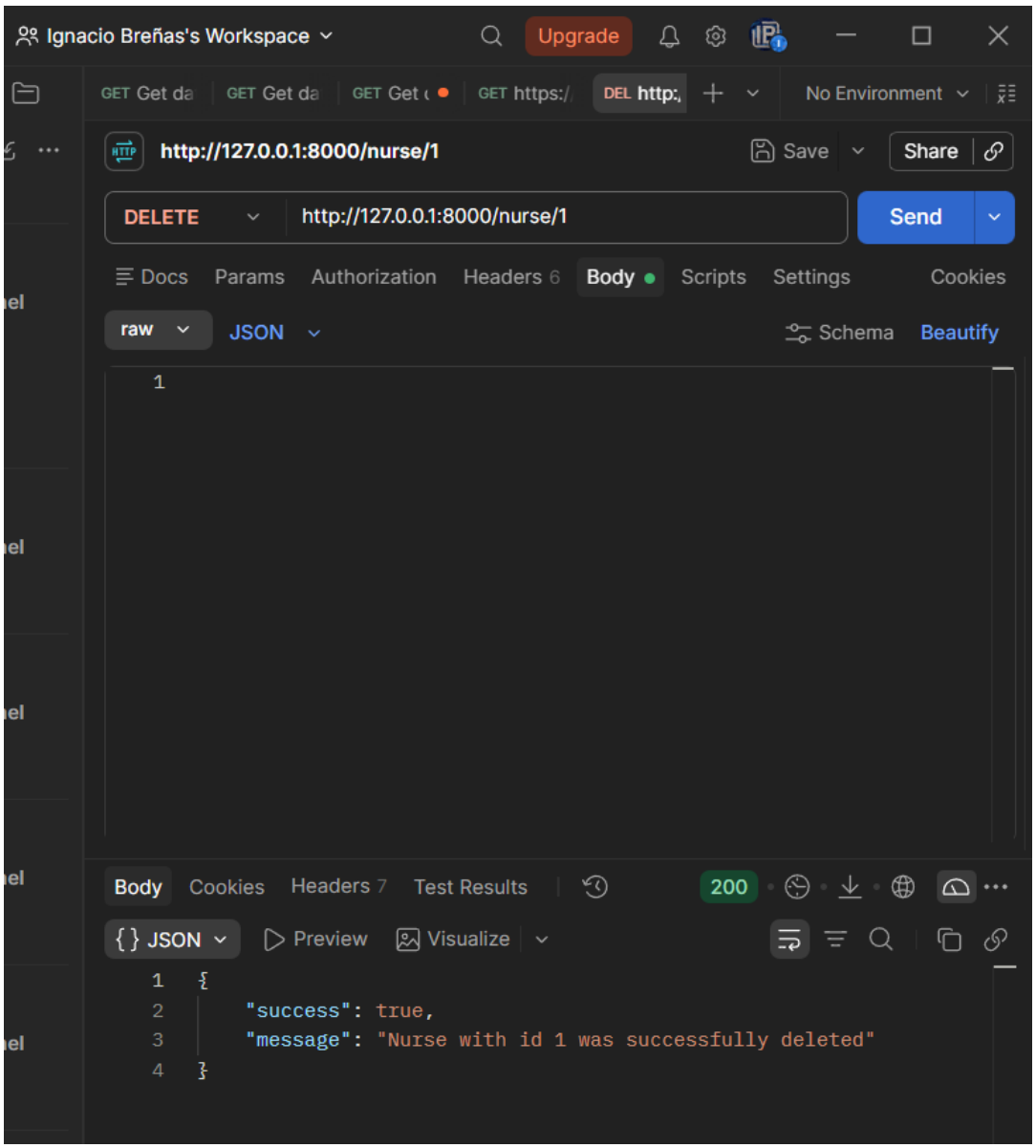
Intento de actualización de un enfermero con ID inexistente (404 Not Found).



*Postman Update (404 Not Found) 1*

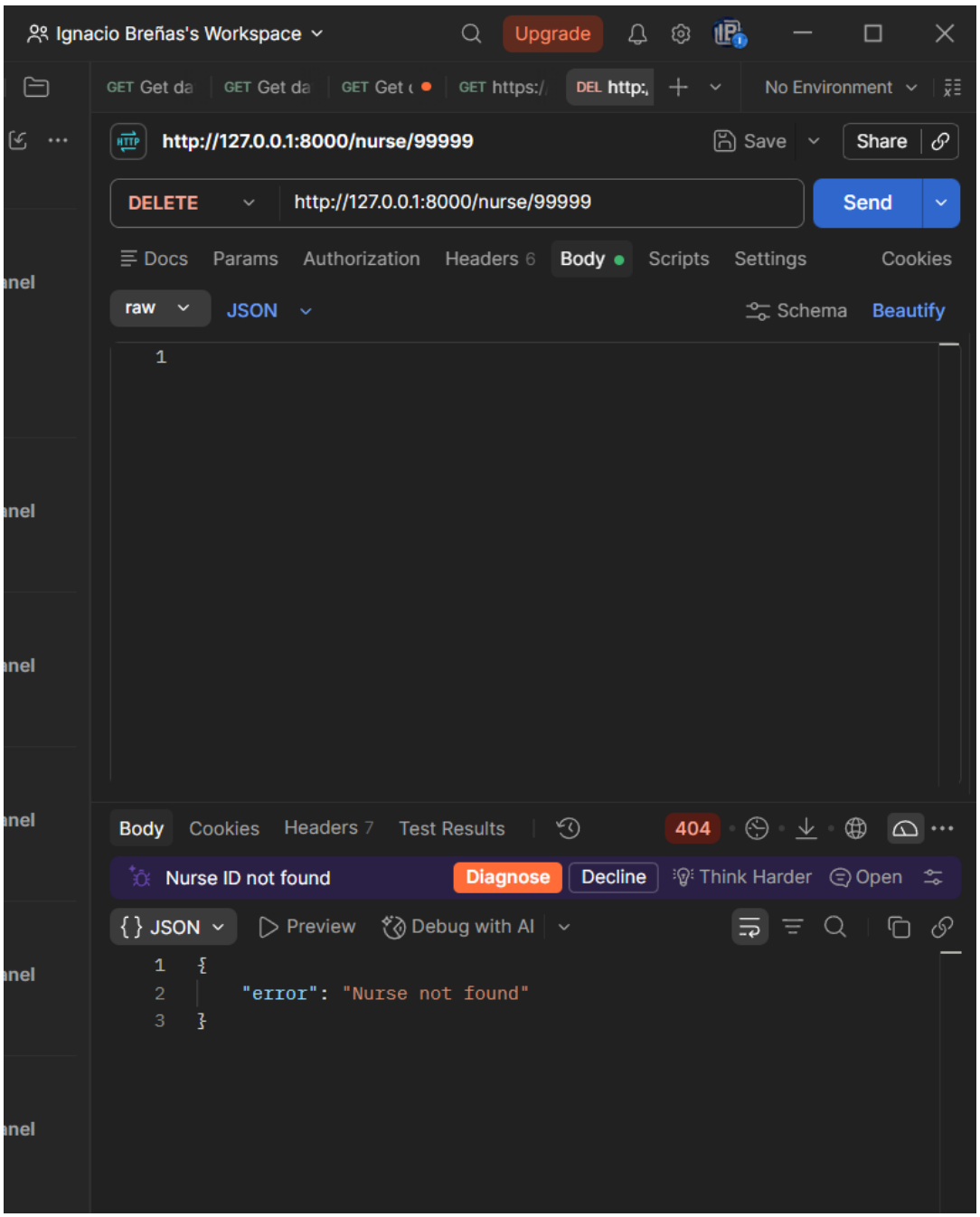
## 6.4. DELETE /nurse/{id} (Delete)

Eliminación física confirmada de un registro de enfermero (200 OK).



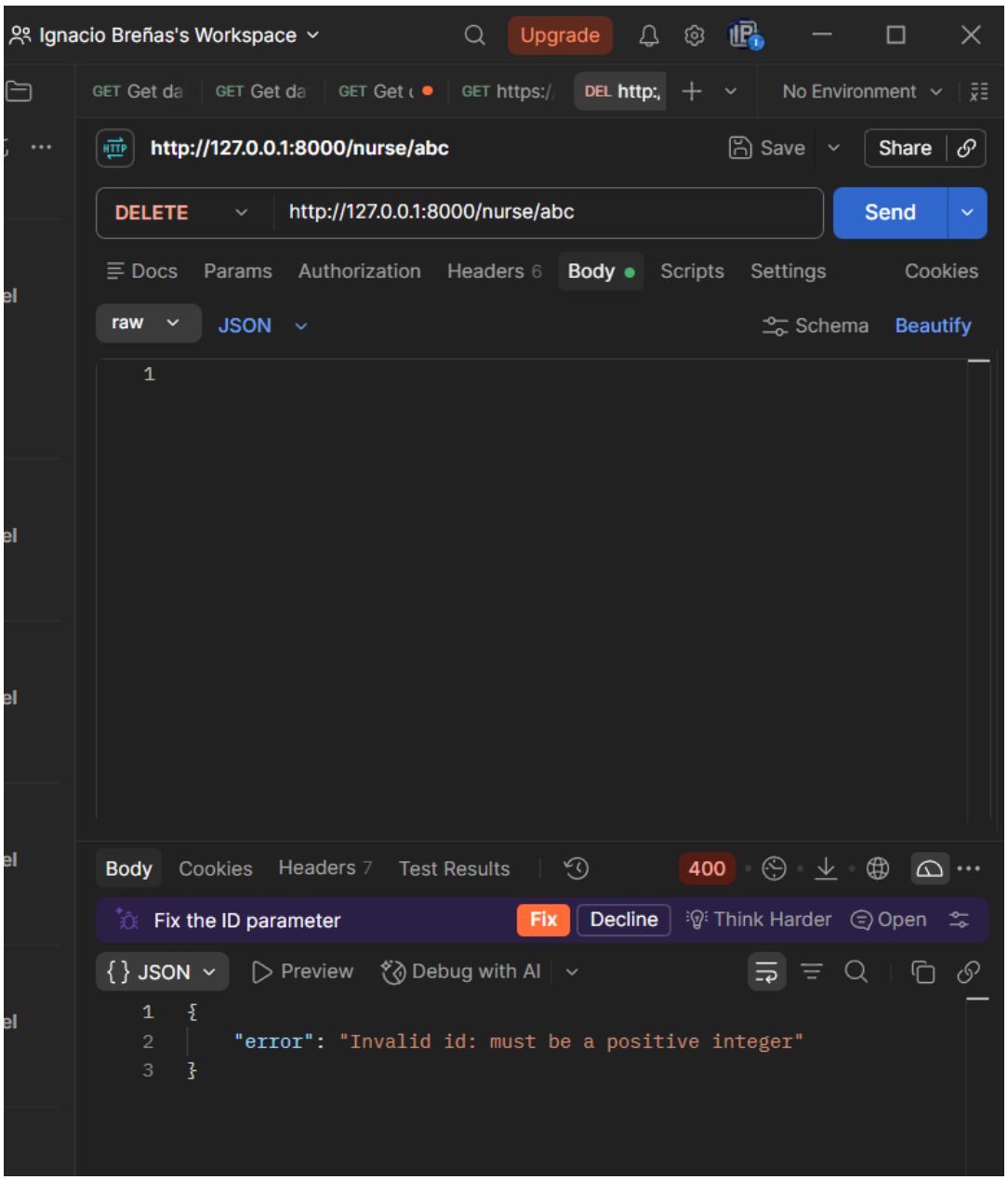

*Postman Delete (200 OK) 1*

Respuesta ante el intento de borrado de un registro inexistente (404 Not Found).




*Postman Delete (404 Not Found) 1*

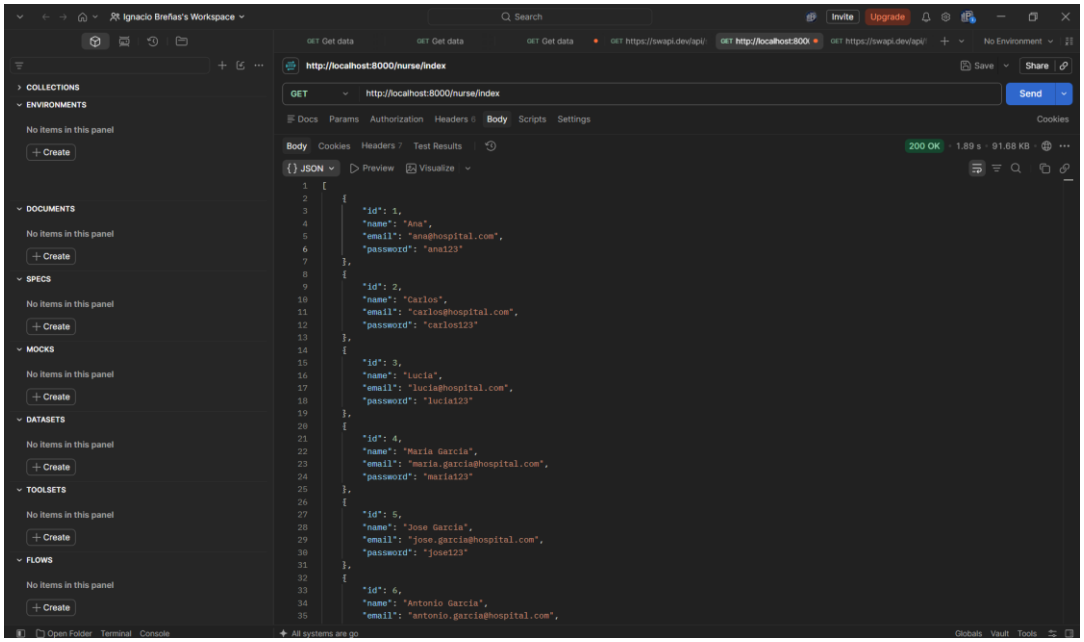
Control de ID no numérico o alfanumérico en la ruta de borrado (400 Bad Request).




*Postman Delete (400 Bad Request) 1*

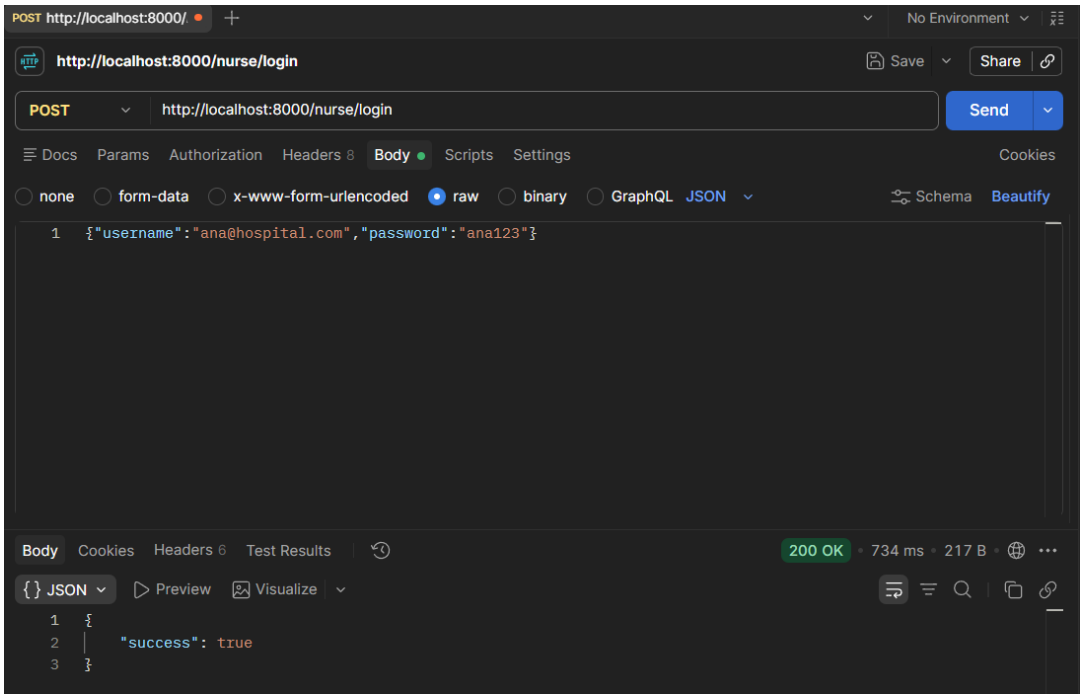
## 6.5. Endpoints ya existentes

Comprobación del listado global de enfermeros desde la base de datos de Supabase.



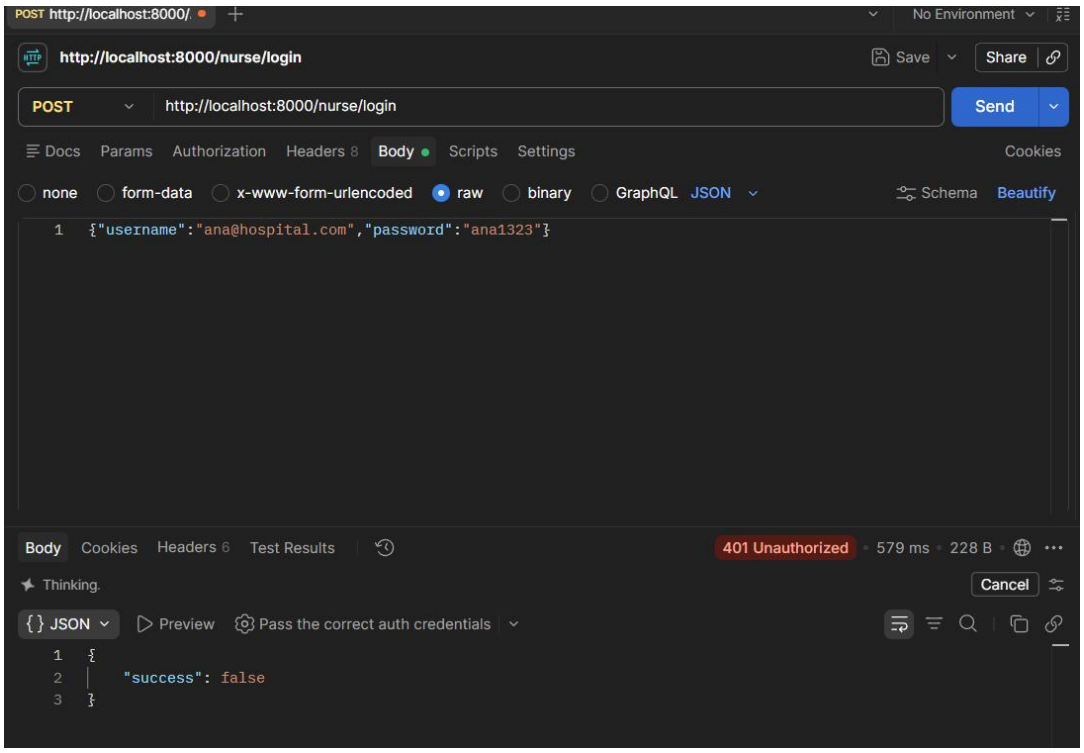

*Postman-Read All 1*

Comprobación del login correcto con contraseña validada contra hash Bcrypt (200 OK).



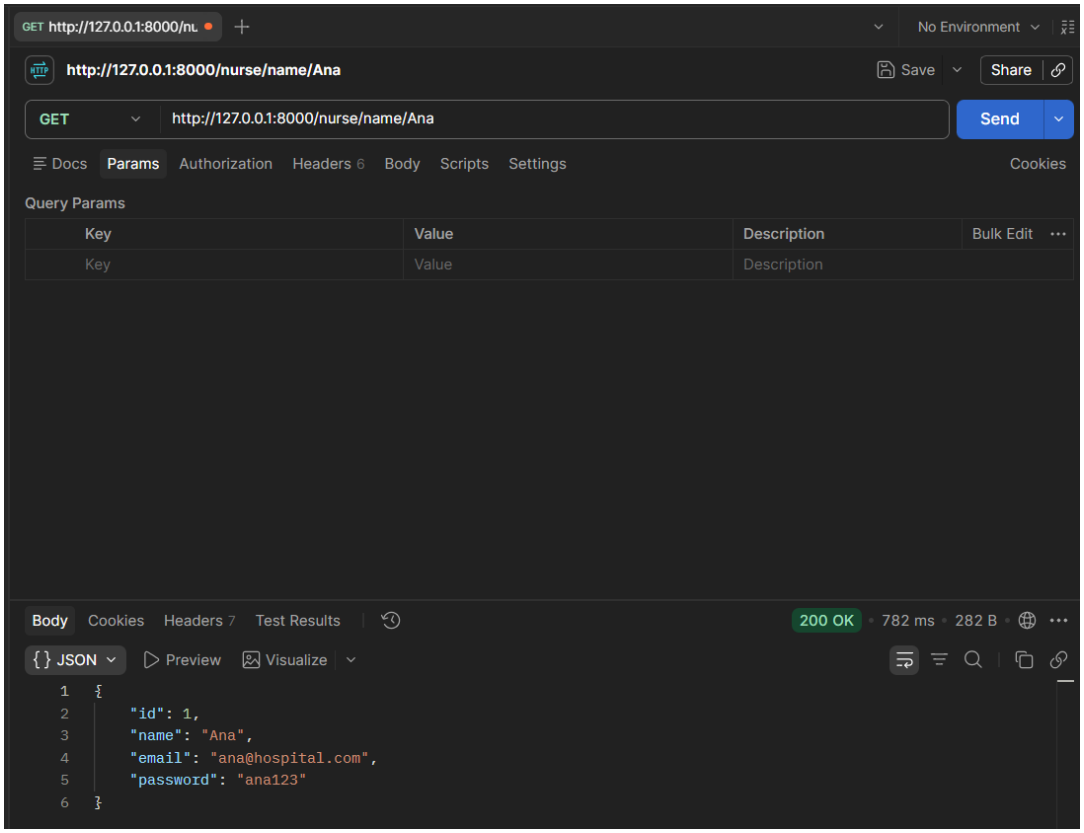

*Postman Login (OK) 1*

Comprobación de login fallido con credenciales incorrectas (401 Unauthorized).



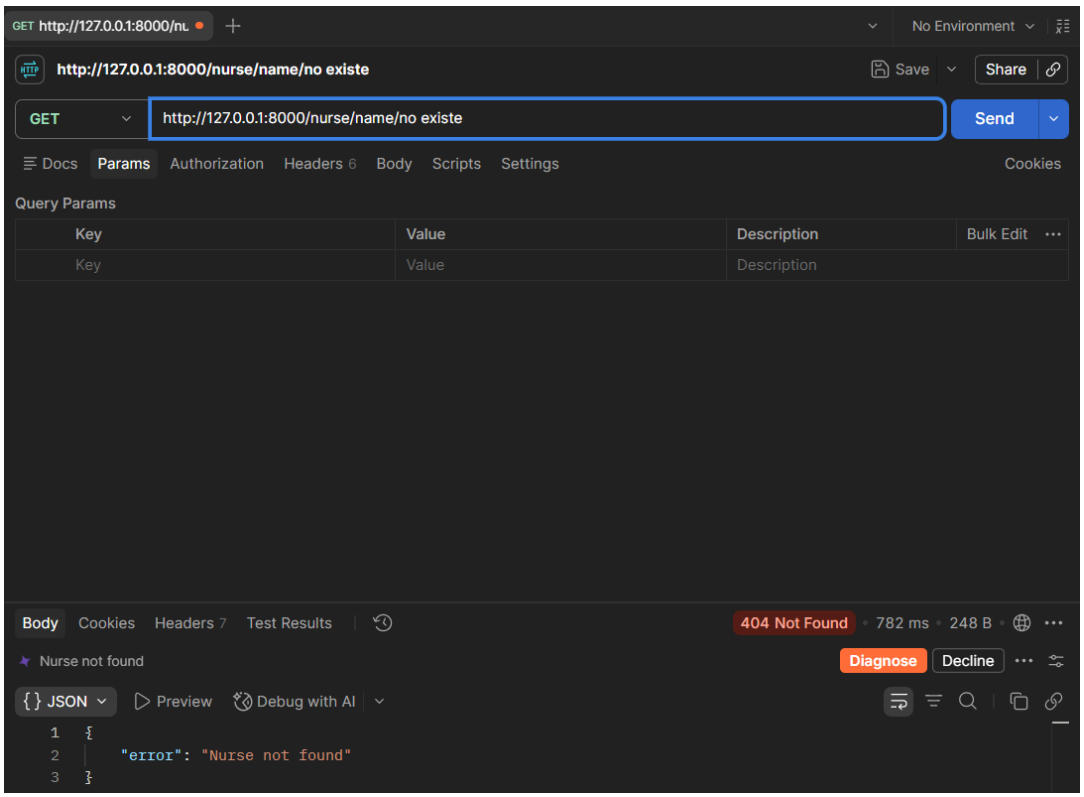

*Postman Login FALLIDO (bad credencials) 1*

Búsqueda de enfermero por nombre coincidente en la base de datos (200 OK).




*Postman ReadByName (OK) 1*

Búsqueda de enfermero por nombre no existente en la base de datos (404 Not Found).




*Postman ReadByName (no existente) 1*